

JavaScript Math Object

In JavaScript, Math is a built-in object that allows you to perform mathematical operations on the Number type. Math is not a constructor function. It's a property of the implicit **global** object.

[JavaScript Math sin\(\)](#)

Returns the sine of an angle

[JavaScript Math cos\(\)](#)

Returns the cosine of an angle

[JavaScript Math tan\(\)](#)

Returns the tangent of an angle

[JavaScript Math.abs\(\)](#)

Returns the absolute value of a number

[JavaScript Math.pow\(\)](#)

Returns a number raised to a certain power

[JavaScript Math.ceil\(\)](#)

Rounds a number up to the next largest integer

[JavaScript Math.floor\(\)](#)

Rounds down a number to the next smallest integer

[JavaScript Math.round\(\)](#)

Returns the number rounded to the nearest integer

[JavaScript Math.trunc\(\)](#)

Returns the integer part of a number

[JavaScript Math.max\(\)](#)

Returns the number with the highest value

[JavaScript Math.min\(\)](#)

Returns the number with the lowest value

[JavaScript Math.sqrt\(\)](#)

Returns the square root of a specified number

[JavaScript Math sign\(\)](#)

Returns the sign of a number

[JavaScript Math.fround\(\)](#)

Returns the 32-bit float representation of number

[JavaScript Math.cbrt\(\)](#)

Returns the cube root of a specified number

[JavaScript Math.random\(\)](#)

Returns pseudo-random float number between 0 and 1

JavaScript Number Object

In JavaScript, Number is a built-in object that allows us to perform various operation on numbers. It consists of different constants and methods to work with numbers.

[JavaScript Number.toString\(\)](#)

Returns a string representation of the number

[JavaScript Number.MAX_SAFE_INTEGER](#)

Represents the maximum safe integer in JavaScript

[JavaScript Number.isSafeInteger\(\)](#)

Checks whether the provided value is safe integer

[JavaScript Number.MIN_SAFE_INTEGER](#)

Represents the minimum safe integer in JavaScript

[JavaScript Number.EPSILON](#)

Represents the smallest floating point number

[JavaScript Number.MIN_VALUE](#)

Represents smallest positive numeric value in JS

[JavaScript Number.MAX_VALUE](#)

Represents the maximum numeric value in JS

[JavaScript Number.NEGATIVE_INFINITY](#)

Represents the negative infinite value

[JavaScript Number.POSITIVE_INFINITY](#)

Represents the positive infinity value

[JavaScript Number.toLocaleString\(\)](#)

Returns specified string representation of number

[JavaScript Number.toExponential\(\)](#)

Returns string exponential notation of number

[JavaScript Number.toPrecision\(\)](#)

Returns string denoting number to given precision

[JavaScript Number.toFixed\(\)](#)

Formats a number using fixed point notation

[JavaScript Number.isInteger\(\)](#)

Checks if the passed value is an integer or not

[JavaScript Number.NaN](#)

Represents Not-A-Number (NaN) value

JavaScript Date and Time

In JavaScript, date and time are represented by the Date object. The Date object provides the date and time information and also provides various methods.

A JavaScript date defines the **EcmaScript epoch** that represents milliseconds since **1 January 1970 UTC**.

Creating Date Objects

There are four ways to create a date object.

- new Date()
- new Date(milliseconds)
- new Date(Date string)
- new Date(year, month, day, hours, minutes, seconds, milliseconds)

new Date()

You can create a date object using the new Date() constructor.

For example,

```
const timeNow = new Date();  
console.log(timeNow); // shows current date and time
```

Here, new Date() creates a new date object with the current date and local time.

new Date(milliseconds)

The Date object contains a number that represents milliseconds since **1 January 1970 UTC**.

new Date(milliseconds) creates a new date object by adding the milliseconds to the zero time.

For example,

```
const time1 = new Date(0);  
console.log(time1); // Thu Jan 01 1970 05:30:00
```

```
const time2 = new Date(1000000000000);  
console.log(time2); // Sat Mar 03 1973 15:16:40
```

Note: 1000 milliseconds is equal to 1 second.

new Date(date string)

new Date(date string) creates a new date object from a date string.

In JavaScript, there are generally three date input formats.

For Example:

```
const date = new Date("2025-10-22");  
console.log(date); // Wed Oct 22 2025 05:45:00 GMT+0545
```

new Date(year, month, day, hours, minutes, seconds, milliseconds)

new Date(year, month,...) creates a new date object by passing specific date and time. For example,

```
const time1 = new Date(2025, 10, 23, 4, 12, 11, 0);  
console.log(time1); // Thu Oct 23 2025 04:12:11
```

JavaScript Date Methods

"Get" Methods (Local Time)

These methods retrieve parts of the date, relative to the user's **local time zone**.

- **getDate()**: Returns the day of the month (1-31).
- **getDay()**: Returns the day of the week (0 for Sunday, 1 for Monday, ..., 6 for Saturday).
- **getFullYear()**: Returns the full four-digit year (e.g., 2025).
- **getHours()**: Returns the hour (0-23).
- **getMilliseconds()**: Returns the milliseconds (0-999).
- **getMinutes()**: Returns the minutes (0-59).
- **getMonth()**: Returns the month (0 for January, 1 for February, ..., 11 for December).
- **getSeconds()**: Returns the seconds (0-59).
- **getTime()**: Returns the number of milliseconds since the UNIX epoch.
- **getTimezoneOffset()**: Returns the time zone difference between UTC and local time, in minutes.

"Set" Methods (Local Time)

These methods modify parts of the date, relative to the user's **local time zone**.

- **setDate(day)**: Sets the day of the month (1-31).
- **setFullYear(year)**: Sets the full four-digit year.
- **setHours(hour)**: Sets the hour (0-23).
- **setMilliseconds(ms)**: Sets the milliseconds (0-999).
- **setMinutes(min)**: Sets the minutes (0-59).
- **setMonth(month)**: Sets the month (0-11).
- **setSeconds(sec)**: Sets the seconds (0-59).
- **setTime(milliseconds)**: Sets the entire date and time using the number of milliseconds since the UNIX epoch.

"Get" Methods (UTC)

These methods retrieve parts of the date, relative to **Coordinated Universal Time (UTC)**.

- **getUTCDate()**: Returns the day of the month in UTC (1-31).
- **getUTCDay()**: Returns the day of the week in UTC (0-6).
- **getUTCFullYear()**: Returns the four-digit year in UTC.
- **getUTCHours()**: Returns the hour in UTC (0-23).
- **getUTCMilliseconds()**: Returns the milliseconds in UTC (0-999).
- **getUTCMinutes()**: Returns the minutes in UTC (0-59).
- **getUTCMonth()**: Returns the month in UTC (0-11).
- **getUTCSeconds()**: Returns the seconds in UTC (0-59).

"Set" Methods (UTC)

These methods modify parts of the date, relative to **Coordinated Universal Time (UTC)**.

- **setUTCDate(day)**: Sets the day of the month in UTC (1-31).
- **setUTCFullYear(year)**: Sets the four-digit year in UTC.
- **setUTCHours(hour)**: Sets the hour in UTC (0-23).
- **setUTCMilliseconds(ms)**: Sets the milliseconds in UTC (0-999).
- **setUTCMinutes(min)**: Sets the minutes in UTC (0-59).
- **setUTCMonth(month)**: Sets the month in UTC (0-11).
- **setUTCSeconds(sec)**: Sets the seconds in UTC (0-59).

Conversion & String Methods

These methods return a string representation of the Date object.

- **toDateString()**: Returns the date portion (day, month, year) as a human-readable string (e.g., "Thu Oct 23 2025").
- **toISOString()**: Returns the date as a string in the simplified ISO 8601 format (e.g., "2025-10-23T03:01:21.123Z").
- **toJSON()**: Returns an `toISOString()` formatted string, primarily for `JSON.stringify()` use.

- **toLocaleDateString():** Returns a language-sensitive string representing the date portion (e.g., "10/23/2025" in the US).
- **toLocaleString():** Returns a language-sensitive string representing the full date and time (e.g., "10/23/2025, 8:31:21 AM").
- **toLocaleTimeString():** Returns a language-sensitive string representing the time portion (e.g., "8:31:21 AM").
- **toString():** Returns the full date and time as a human-readable string, including time zone (e.g., "Thu Oct 23 2025 08:31:21 GMT+0530 (India Standard Time)").
- **toTimeString():** Returns the time portion as a human-readable string, including time zone (e.g., "08:31:21 GMT+0530 (India Standard Time)").
- **toUTCString():** Returns the date and time as a string formatted according to UTC conventions (e.g., "Thu, 23 Oct 2025 03:01:21 GMT").

JavaScript Objects

JavaScript object is a variable that can store multiple data in **key-value** pairs.

You can create an object in three different ways:

1. Using object literal
2. By creating instance of Object directly
3. By using constructor function

Using object literal

```
// program to create JavaScript object using object literal
const person = {
  name: 'John',
  age: 20,
  hobbies: ['reading', 'games', 'coding'],
  score: {
    maths: 90,
    science: 80
  }
};
```

Create an Object using Instance of Object Directly

The syntax for creating an object using instance of an object is:

```
const objectName = new Object();
```

```
// program to create JavaScript object using instance of an object
```

```
const person = new Object ( {  
  name: 'John',  
  age: 20,  
  hobbies: ['reading', 'games', 'coding'],  
  score: {  
    maths: 90,  
    science: 80  
  }  
});
```

Create an object using Constructor Function

```
function Person() {  
  this.name = 'John',  
  this.age = 20,  
  this.hobbies = ['reading', 'games', 'coding'],  
  this.score = {  
    maths: 90,  
    science: 80  
  }  
}  
  
const person = new Person();  
console.log(typeof person); // object
```

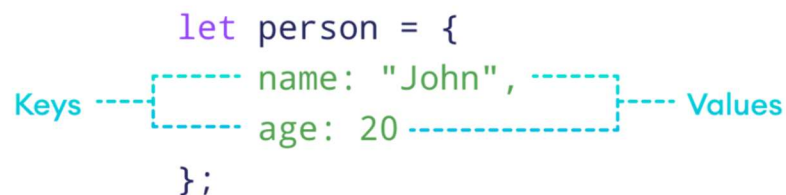
JavaScript Object Properties

In JavaScript, the key-value pairs of an object are referred to as **properties**. For example,

```
const person = {  
  name: "John",  
  age: 20,  
};
```

Here, name: "John" and age: 30 are the properties of the object person.

```
let person = {  
  name: "John",  
  age: 20  
};
```



Access Object Properties

You can access the **value** of a property by using its **key**.

1. Using Dot Notation

```
const dog = {  
  name: "Rocky",  
};  
  
// access property  
console.log(dog.name);  
  
// Output: Rocky
```

2. Using Bracket Notation

```
const cat = {  
  name: "Luna",  
};  
  
// access property  
console.log(cat["name"]);  
  
// Output: Luna
```

JavaScript Object Operations

In JavaScript, we can perform various operations on object properties like modifying, adding, deleting.

1. Modify Object Properties

We can modify object properties by assigning a new value to an existing key.

For example,

```
const person = {  
  name: "Bobby",  
  hobby: "Dancing",  
};  
// modify property  
person.hobby = "Singing";  
  
// display the object  
console.log(person);  
// Output: { name: 'Bobby', hobby: 'Singing' }
```

2. Add Object Properties

```
const student = {  
  name: "John",  
  age: 20,  
}  
// add properties  
student.rollNo = 14;  
student.faculty = "Science";  
// display the object  
console.log(student);  
// Output: { name: 'John', age: 20, rollNo: 14, faculty: 'Science' }
```

3. Delete Object Properties

We can remove properties from an object using the delete operator. For example,

```
const employee = {  
  name: "Tony",  
  position: "Officer",  
  salary: 30000,  
};  
// delete object property  
delete employee.salary  
// display the object  
console.log(employee);  
// Output: { name: 'Tony', position: 'Officer' }
```

JavaScript Nested Objects

A nested object contains another object as a property. For example,

```
// outer object student  
const student = {  
  name: "John",  
  age: 20,  
  // contains another object marks  
  marks: {  
    science: 70,  
    math: 75  
  }  
};  
  
// display student  
console.log(student);  
// Output: { name: 'John', age: 20, marks: { science: 70, math: 75 } }.
```

Access Properties of Nested Objects

We can access a nested object's property using both the dot and bracket notations.

```
const student = {  
  name: "John",  
  age: 20,  
  marks: {  
    science: 70,  
    math: 75  
  }  
};  
  
// use dot notation  
console.log(student.marks.science); // 70  
  
// use bracket notation  
console.log(student["marks"]["math"]); // 75
```

JavaScript Object Methods

We can also include functions inside an object. For example,

```
const person = {  
  name: "Bob",  
  age: 30,  
  // use function as value  
  greet: function () {  
    console.log("Bob says Hi!");  
  }  
};  
  
// call object method  
person.greet(); // Bob says Hi!
```

These functions that are defined inside objects are called **methods**.

Note: Just like we use () to call a function, we must use () to call methods.

JavaScript this Keyword

We use this keyword in an object method to access a property of the same object. For example,

```
// person object
const person = {
  name: "John",
  age: 30,

  // method
  introduce: function () {
    console.log(`My name is ${this.name} and I'm ${this.age} years old.`);
  }
};

// access the introduce() method
person.introduce();

// Output: My name is John and I'm 30 years old.
```

JavaScript Constructor Function

In JavaScript, a constructor function is used to create and initialize [objects](#).

Example

```
// constructor function
function Person () {
  this.name = "John",
  this.age = 23
}

// create an object
const person = new Person();
```

```
// print object attributes
console.log(person.name);
console.log(person.age);
```

```
// Output:
// John
// 23
```

Create Multiple Objects With Constructor Function

In JavaScript, you can create multiple objects from a constructor function. For example,

```
// constructor function
```

```
function Person () {
    this.name = "John",
    this.age = 23,

    this.greet = function () {
        console.log("hello");
    }
}
```

```
// create objects
```

```
const person1 = new Person();
const person2 = new Person();
```

```
// access properties
```

```
console.log(person1.name); // John
console.log(person2.name); // John
```

JavaScript Constructor Function Parameters

You can also create a constructor function with parameters. For example,

```
// constructor function with parameters
function Person (person_name, person_age, person_gender) {

    // assign parameter values to the calling object
    this.name = person_name,
    this.age = person_age,
    this.gender = person_gender,

    this.greet = function () {
        return (`Hi ${this.name}`);
    }
}

// create objects and pass arguments
const person1 = new Person("John", 23, "male");
const person2 = new Person("Sam", 25, "female");

// access properties
console.log(person1.name); // John
console.log(person2.name); // Sam
```

JavaScript Getter and Setter

In JavaScript, there are two kinds of object properties:

- Data properties
- Accessor properties

Data Property

```
const student = {  
  
  // data property  
  firstName: 'Monica';  
};
```

Accessor Property

In JavaScript, accessor properties are [methods](#) that get or set the value of an [object](#).

we use these two [keywords](#):

- get - to define a getter method to get the property value
- set - to define a setter method to set the property value

JavaScript Getter

In JavaScript, getter methods are used to access the properties of an object.
For example,

```
const student = {  
  
  // data property  
  firstName: 'Monica',  
  
  // accessor property(getter)  
  get getName() {  
    return this.firstName;  
  }  
};  
  
// accessing data property  
console.log(student.firstName); // Monica  
  
// accessing getter methods  
console.log(student.getName); // Monica  
  
// trying to access as a method  
console.log(student.getName()); // error
```

JavaScript Setter

In JavaScript, setter methods are used to change the values of an object. For example,

```
const student = {  
  firstName: 'Monica',  
  //accessor property(setter)  
  set changeName(newName) {  
    this.firstName = newName;  
  }  
};  
  
console.log(student.firstName); // Monica  
  
// change(set) object property using a setter  
student.changeName = 'Sarah';  
  
console.log(student.firstName); // Sarah
```

JavaScript Prototype

In JavaScript, prototypes allow properties and methods to be shared among instances of the function or object. For example,

```
function Car() {  
  console.log("Car instance created!");  
};  
  
// add a property to prototype  
Car.prototype.color = "Red";  
  
// add a method to the prototype  
Car.prototype.drive = function () {  
  console.log(`Driving the car painted in ${this.color}...`);  
};
```

```
// display the added property  
console.log(`The car's color is: ${Car.prototype.color}`);
```

```
// call the added method  
Car.prototype.drive();
```

Output

The car's color is: Red

Driving the car painted in Red...

Javascript Object Methods

- **Object.assign(target, ...sources)**: Copies all properties from one or more source objects to a target object. It returns the modified target object.
- **Object.create(obj)**: Creates a new object with the specified prototype object.
- **Object.defineProperty(obj, prop, descriptor)**: Defines a new property or modifies an existing property on an object.
- **Object.defineProperties(obj, props)**: Defines or modifies multiple properties on an object at once.
- **Object.entries(obj)**: Returns an array containing an object's string-keyed property [key, value] pairs.
- **Object.freeze(obj)**: "Freezes" an object. This prevents new properties from being added, existing properties from being removed, and the attributes or values of existing properties from being changed.
- **Object.fromEntries(iterable)**: Transforms a list of [key, value] pairs (like those from Object.entries) back into a new object.
- **Object.isExtensible(obj)**: Determines if new properties can be added to an object.
- **Object.isFrozen(obj)**: Determines if an object is frozen (i.e., not extensible, and all properties are non-configurable and non-writable).

- **Object.isSealed(obj):** Determines if an object is sealed .
- **Object.keys(obj):** Returns an array of an object's own *enumerable* string-keyed property names.
- **Object.preventExtensions(obj):** Prevents any new properties from ever being added to an object.
- **Object.seal(obj):** "Seals" an object. This prevents new properties from being an object
- **Object.values(obj):** Returns an array of an object's own *enumerable* string-keyed property values.